

AGENTIC AI SOLUTIONS ENGINEER — WAREHOUSE AUTOMATION

Phase 0 Design Document

Warehouse Ops — Order Exception Resolver + Shift Planner

Repository	https://github.com/sunkaramahesh09/warehouse-agent
Live app	https://app-production-fd3e.up.railway.app
Source	<code>docs/PHASE_0_DESIGN.md</code>
Generated	2026-10-01

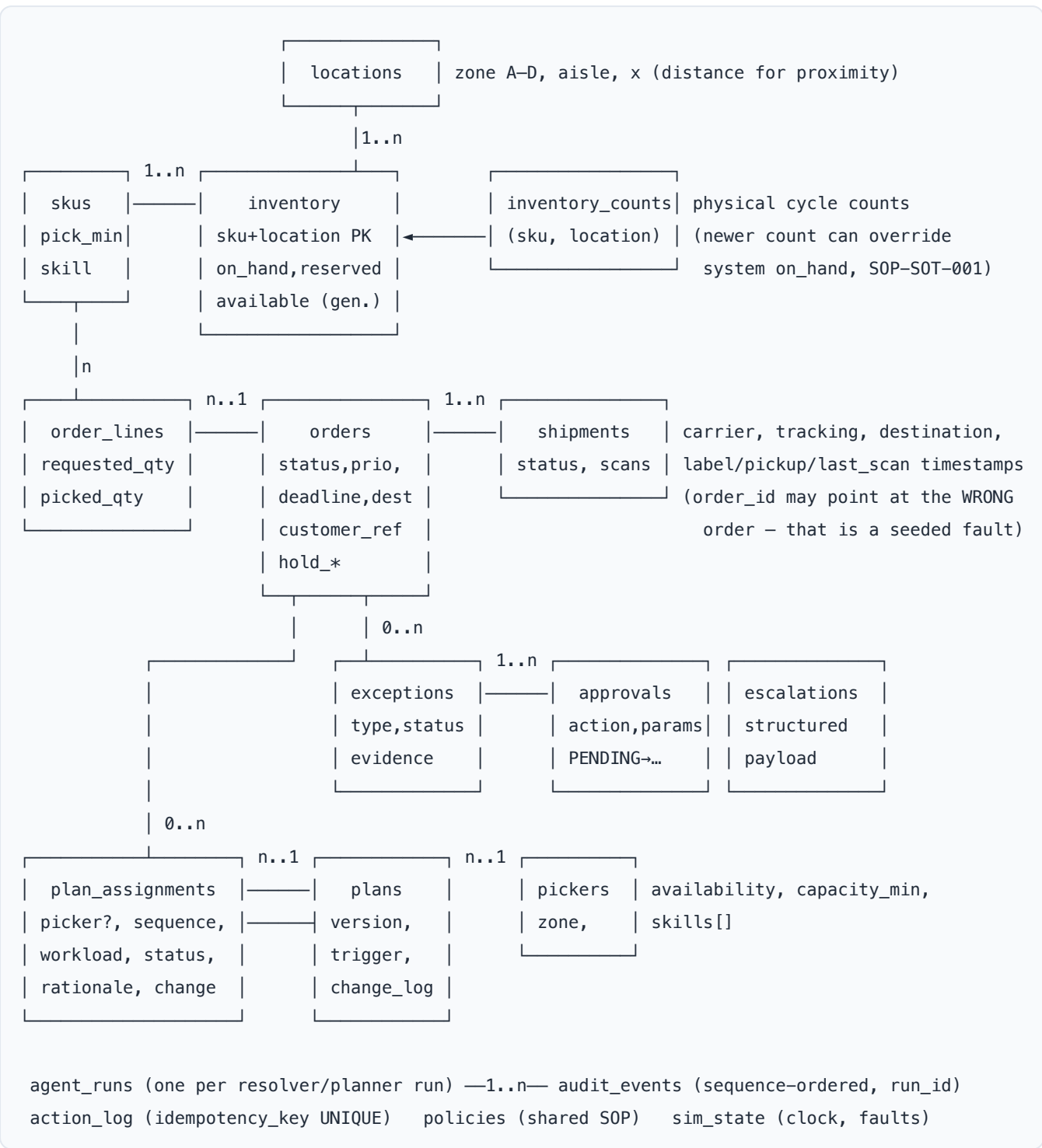
Simulated prototype: all data is fictional; nothing connects to a real warehouse, carrier or customer system.

Phase 0 — Design Before Code

Written before implementation began (2026-09-30) and then kept in sync with the code. Where the implementation deviated from this plan, the change is recorded in [§9 Changes from the original plan](#) and in `REFLECTION.md`.

System: **Order Exception Resolver + Shift Task Planner** operating on one simulated WMS/OMS database ("the environment"). Everything is fictional and every action is **simulated**. Nothing connects to a real warehouse, carrier, or customer.

1. Entity model and relationships



Entity	Key fields	Used by investigation	Used by planning
orders	order_id, status, priority (1=highest), created_at, deadline, destination_ref, customer_ref, assigned_picker_id, hold_reason, hold_exception_id, hold_prev_status	status vs shipment, destination vs shipment, duplicate criteria, timestamp sanity	eligibility (status/hold), deadline urgency, priority, in-progress picker
order_lines	order_id, sku, requested_qty, picked_qty, status	invalid quantities, "fully picked?" for status sync, shortfall	remaining workload = (requested – picked) x sku.pick_minutes

Entity	Key fields	Used by investigation	Used by planning
skus	sku, description, pick_minutes_per_unit, required_skill	—	workload, skill constraint (COLD, BULKY)
inventory	(sku, location_id) PK, on_hand, reserved, available = on_hand – reserved (generated column), last_updated	shortfall, missing location	inventory readiness gate, allocation in priority order
inventory_counts	sku, location_id, counted_qty, counted_at	mismatch between system and physical count	(not read by the planner — the resolver turns a confirmed mismatch into an order hold)
shipments	shipment_id, order_id, status, carrier, tracking_ref, destination_ref, label_created_at, picked_up_at, last_scan_at	desync, stale shipment, wrong link, destination conflict	—
pickers	picker_id, display_name, availability, capacity_minutes, home_zone, skills[]	—	availability, capacity, skills, location tie-break
exceptions	exception_id, order_id, type, status, summary, evidence_refs (json), detected_at, resolution (json)	subject of each resolver run	shown as warnings/blocks via order hold
approvals	approval_id, exception_id, action_type, params, effect, policy_ids, status, idempotency_key, expires_at, decided_by	confirmation gate	—
escalations	escalation_id, exception_id, payload (structured, §4.3), status, resolution_note	escalation queue	—
plans / plan_assignments	version, trigger, parent_version, change_log / order_id, picker_id, sequence, status, workload_min, est_start/finish, readiness, rationale, change_type	—	plan output + version history
audit_events	seq, run_id, ts, workflow, event_type, tool_name, input, result, error, policy_refs, decision_summary, proposed_action, approval_state, state_changes, outcome	trace	trace
action_log	idempotency_key (unique), action, result	duplicate-action protection	duplicate-dispatch protection
sim_state	sim_now, shift_start, shift_end, active_faults	staleness age	"now", remaining shift

2. Data assumptions and source-of-truth choices

- 1. Simulated clock.** All ages and deadlines are computed from `sim_state.sim_now`, not the wall clock. Baseline: `2026-10-01 08:00 UTC`, shift ends `16:00`. This makes every scenario reproducible.
- 2. Inventory.** The `inventory` table is the system of record. `available = on_hand – reserved` is a generated column, so it cannot drift from its parts. A physical cycle count (`inventory_counts`) **newer than** `inventory.last_updated` supersedes system `on_hand` for decisions. The system never *raises* stock on its own authority. Correcting `on_hand` is an inventory adjustment, which is a human action (SOP-SOT-001).
- 3. Shipment movement.** Carrier scan events (`picked_up_at`, `last_scan_at`) are authoritative for physical movement. A shipment record or label alone is **not** evidence of shipment. An order may be moved forward to `SHIPPED` only when all three hold: (a) a carrier pickup scan exists, (b) every line is fully picked, (c) the shipment destination equals the order destination (SOP-SOT-002).

4. **Order status** is owned by the OMS (`orders.status`) and follows an explicit transition table (§3.2).
The system never moves an order status *backwards* on its own.
5. **Neither record is authoritative for destination.** If the order destination and the shipment destination differ, no rule establishes precedence. The case is ambiguous by definition and must be escalated (SOP-EXC-005).
6. **Workload unit = minutes.** Order workload = $\sum \text{remaining_qty} \times \text{pick_minutes_per_unit} + 2 \text{ min per distinct location} + 3 \text{ min per extra zone visited}$. Picker `capacity_minutes` is the remaining productive time this shift.
7. **Orders are not split across pickers** (a single picker owns an order). This is a simplification, stated in SOP-PLN-002.
8. **Duplicate criterion:** same `customer_ref` , same `destination_ref` , identical multiset of (sku, qty), and created within 30 minutes. The later order is the suspected duplicate.
9. **Staleness:** a shipment in `LABEL_CREATED` with no carrier scan for more than 48 h is stale.

3. Statuses and allowed transitions

3.1 Order statuses

`PENDING` (released, not started) · `PICKING` (in progress, has picker) · `PICKED` · `PACKED` · `SHIPPED` · `ON_HOLD` · `CANCELLED`

3.2 Allowed transitions (enforced in code by `assertTransition`)

From	To
PENDING	PICKING, ON_HOLD, CANCELLED
PICKING	PICKED, ON_HOLD
PICKED	PACKED, ON_HOLD
PACKED	SHIPPED, ON_HOLD
ON_HOLD	<i>previous status</i> (release), CANCELLED
SHIPPED, CANCELLED	— (terminal)

Any other transition returns `INVALID_TRANSITION` and nothing is written.

3.3 Exception statuses

`OPEN` → `INVESTIGATING` → {`RESOLVED` | `AWAITING_APPROVAL` | `ESCALATED` | `FAILED`} ; `AWAITING_APPROVAL` → {`RESOLVED` (approved+executed) | `ESCALATED` (rejected/expired)} ; `ESCALATED` → `CLOSED` (reviewer records resolution).

4. Seeded exception types, triggers, and action boundaries

ID	Type	Seeded trigger (reproducible)	Boundary	Rationale
EXC-2001	INVENTORY_SHORTFALL	ORD-1004 needs 12 × SKU-007. System says 14 on hand, but a cycle count at 07:30 (newer than <code>last_updated</code>) found 5	Autonomous hold + escalation for replenishment	A hold is reversible and prevents picking an unfulfillable order. Stock is never invented and quantities are never reduced
EXC-2002	DUPLICATE_ORDER	ORD-1011 matches ORD-1010: same customer, destination, and lines, created 3 min apart	Autonomous hold of the later order + cancel requires explicit operator approval	Cancelling is irreversible and has customer impact (SOP-EXC-002, SOP-APR-001)
EXC-2003	STATUS_DESYNC (recoverable)	ORD-1003 is <code>PACKED</code> . SHP-5003 has a carrier pickup scan and an in-transit scan, all lines are fully picked, destination matches	Autonomous forward sync <code>PACKED</code> → <code>SHIPPED</code>	All three SOP-SOT-002 conditions are met, so the evidence is corroborated and not just "a shipment exists"
EXC-2004	STATUS_DESYNC (contradictory)	ORD-1008 is <code>SHIPPED</code> , but SHP-5008 is <code>LABEL_CREATED</code> with no scan and one line is only partially picked	Escalate, preserve state	Correcting it would mean a backward move, and the records contradict each other
EXC-2005	INVALID_DATA	ORD-1009 line 2 has <code>requested_qty = -3</code> and <code>created_at</code> is after its <code>deadline</code>	Autonomous hold + escalate. Never auto-correct	The intended quantity is unknowable, so any correction would be invented
EXC-2006	STALE_SHIPMENT	ORD-1007 is <code>PACKED</code> . SHP-5007 has been <code>LABEL_CREATED</code> for 70 h with no scan	Escalate (carrier follow-up). No status change	No carrier tool exists and no evidence supports any state change
EXC-2007	DESTINATION_CONFLICT / wrong link — ambiguous	SHP-5012 is linked to ORD-1012 (DEST-W-212), but its destination is DEST-E-340, which is ORD-1013's destination. ORD-1013 has no shipment	No autonomous relink/cancel. Hold ORD-1012 + escalate with unresolved questions	Either the link is wrong or the order address changed. The data cannot say which (SOP-EXC-005)

Planning-side seeded constraints (no exception record, detected by the planner itself): * **ORD-1015**: SKU-010 × 20 requested, only 8 available → `BLOCKED` (`inventory not ready`) . * **ORD-1014**: bulky order whose workload exceeds every eligible picker's capacity → `INFEASIBLE` (`capacity`) . * **ORD-1006**: a COLD item. Only P-02 has the COLD skill, so when P-02 goes unavailable it becomes `INFEASIBLE` (`skill`) . * **P-05** is unavailable from the start, so it must never receive work. * **ORD-1005**, **ORD-1018** are already `PICKING` (existing progress), so they stay with their picker.

4.1 Decision outcomes

`AUTO_RESOLVED` (permitted action executed and verified) · `AWAITING_APPROVAL` (proposal created, nothing executed) · `ESCALATED` (state preserved or safely held, structured escalation) · `NO_ACTION_NEEDED`

(evidence shows the issue no longer exists) · **FAILED** (tool/LLM error, with explicit state and no success claimed)

4.2 Action authority (SOP-APR-001, machine-readable, read by the guard)

- **Autonomous:** `HOLD_ORDER` , `SYNC_ORDER_STATUS_FORWARD` (only under SOP-SOT-002), `CREATE_ESCALATION` , `REQUEST_APPROVAL`
- **Approval required:** `CANCEL_ORDER` , `RELINK_SHIPMENT` , `ADJUST_INVENTORY` , `RELEASE_HOLD`
- **Prohibited for the agent:** `DELETE_ORDER` , `MERGE_ORDERS` (not implemented as a tool at all), backward status changes

4.3 Structured escalation payload

```
exception_id, order_id, detected_issue, evidence_checked[], tool_results[],  
conflicting_facts[], missing_facts[], policy_refs[], recommended_human_action,  
actions_already_taken[], current_state, unresolved_questions[]
```

5. Planning policy (stated before implementation)

Step 0: Pinning (existing progress). Orders already `PICKING` stay with their current picker if that picker is available. Only remaining quantity counts as workload. Completed work is never re-planned.

Step 1: Eligibility gates (hard), evaluated in order. The first failure is reported: 1. Status must be `PENDING` or `PICKING`. `ON_HOLD` → **BLOCKED** with `hold_exception_id` and reason (this is the cross-agent link). 2. Data validity: every line has `requested_qty > 0` and $0 \leq \text{picked_qty} \leq \text{requested_qty}$ → otherwise **BLOCKED (invalid data)**. 3. Inventory readiness: remaining qty per SKU must be coverable from `available`, allocated in priority order (higher-priority orders consume stock first) → otherwise **BLOCKED (inventory)**.

Step 2: Priority order (deterministic sort key): 1. Pinned in-progress work first. 2. **Urgency bucket** by time-to-deadline from `sim_now`: $\leq 2\text{ h} \rightarrow 0$, $\leq 4\text{ h} \rightarrow 1$, within shift → 2, after shift → 3 (already overdue → 0). 3. Explicit `priority` (1 before 2 before 3). 4. Earlier `deadline`. 5. Earlier `created_at`, then `order_id` (total order, so output is deterministic).

So SLA urgency beats explicit priority only across buckets. Within a bucket, explicit priority wins. Rationale: a P3 order due in 90 min is more urgent than a P1 due at end of shift, but between two orders due in the same window the business priority decides.

Step 3: Assignment (greedy, per order in priority order). Candidate pickers must be `AVAILABLE`, have the required skills, and satisfy `load + workload ≤ capacity_minutes` (**hard**). Among the candidates: 1. Prefer pickers who finish **before the deadline** (`est_finish = shift_now + load + workload`). 2. Then (replanning only) prefer the order's **previous picker** (stability). 3. Then earliest `est_finish`. 4. Then **location proximity** (zone distance between the picker's current zone and the order's primary zone). This is the secondary factor. 5. Then `picker_id`.

If a feasible picker exists but none meets the deadline → **ASSIGNED with SLA_AT_RISK** (late is better than never, and it is flagged). If no picker has enough capacity → **INFEASIBLE (capacity)**. If no available picker has the skill → **INFEASIBLE (skill)**.

Step 4: Replanning (incremental, never from zero). Keep `COMPLETED` and still-valid `IN_PROGRESS` assignments frozen. Invalidate assignments whose picker became unavailable, or whose order is now held or no longer inventory-ready. Re-run Steps 1–3 for all un-started work with stickiness to the previous picker. Record a new plan version with a per-order `change_type` (`KEPT` , `RESEQUENCED` , `REASSIGNED` , `NEW` , `NEWLY_BLOCKED` , `NEWLY_INFEASIBLE` , `UNBLOCKED` , `COMPLETED_KEPT` , `IN_PROGRESS_KEPT` , `PROGRESS_PRESERVED_REASSIGNED`) and a reason.

Infeasible ≠ error. A planner run that completes with infeasible orders is status `COMPLETED` . A planner exception or timeout is `FAILED` : no new version is written and the previous active plan stays active.

The parameters (bucket thresholds, travel minutes) live in the SOP records (`SOP-PLN-001/002`) and are read by the planner at run time.

6. Agent architecture (summary; see ARCHITECTURE.md)

- **Exception Investigator** calls read-only tools. It is either LLM-driven (Gemini function calling) or deterministic (step policy chosen from prior results). Every tool result goes into an **evidence ledger**.
- **Decision flow** takes the LLM's proposal (`submit_decision`), then a deterministic **assessment** that recomputes the facts from the ledger, then a **policy guard**. The guard checks that the action is in the permitted set, that the evidence is complete, and that cited policy IDs exist and were retrieved in this run. It picks the more conservative of LLM and assessment. Then the **controlled action tool** runs, the new state is verified by re-reading it, and the result is audited.
- **Shift Planner** takes a snapshot through read tools, runs the pure deterministic planner, persists the plan through the `create_plan` tool, and the LLM (optional) explains it using only the computed numbers.
- The LLM never gets SQL, never executes approvals, and its narrative never replaces tool results: action statuses in the report come from tool results.

7. Failure modes and mitigations (system-specific)

#	Failure	Where it bites	Mitigation	Evidence
F1	Missing record (exception references ORD that doesn't exist, shipment link to nothing)	investigation	Read tools return <code>NOT_FOUND</code> explicitly. Assessment treats missing required evidence as a reason to escalate and never fills defaults	scenario <code>missing-record</code>
F2	Malformed record (negative qty, <code>created_at > deadline</code>)	both	Zod validation on tool I/O, DB CHECK where it would not hide seeded faults, planner validity gate, no auto-correction	EXC-2005, planner test
F3	Tool timeout / simulated API failure during an action	resolver	Fault injection (<code>sim_state.faults</code>). The action result is <code>TIMEOUT</code> , then the orchestrator re-reads the target to verify, then reports "status unknown → verified not applied". Exception → <code>FAILED</code> , nothing is claimed	scenario <code>tool-timeout</code>
F4	Duplicate action (double-click approve, re-run investigation)	both	<code>action_log.idempotency_key</code> UNIQUE. Atomic UPDATE approvals ... WHERE <code>status='APPROVED'</code> . A duplicate returns the prior result with <code>duplicate=true</code> and an audit <code>DUPLICATE_SUPPRESSED</code>	test <code>duplicate action</code>

#	Failure	Where it bites	Mitigation	Evidence
F5	Approval not received / expired / rejected	resolver	Action not executed. The exception stays <code>AWAITING_APPROVAL</code> , or on reject/expiry becomes <code>ESCALATED</code> . Executing without <code>APPROVED</code> → <code>APPROVAL_REQUIRED</code>	test + scenario
F6	Picker becomes unavailable mid-shift	planner	Replan invalidates that picker's un-started and in-progress work, preserves picked quantities, and reassigns under capacity and skill	scenario replanning
F7	Inventory changed between plan generation and dispatch	planner	<code>dispatch_assignment</code> re-checks availability at start time. On shortfall → assignment <code>BLOCKED</code> (inventory changed since plan vN) , audit, replan suggested	scenario inventory-drift
F8	LLM misbehaviour (hallucinated policy ID, proposes cancel autonomously, invalid JSON, 429 rate limit)	resolver	Guard validates citations and boundaries. Bad tool args are rejected by Zod and returned to the model. 429 → backoff, then deterministic fallback, labelled in the run	tests on guard
F9	Planner error vs infeasible	planner	Separate run statuses (<code>FAILED</code> vs <code>COMPLETED</code> with infeasible list). The previous plan stays active	scenario planner-failure

8. Shared policy / SOP set

Stored once (`server/src/policy/sop.json` → `policies` table). Both workflows retrieve it through the same `search_policies` / `get_policy` tools and read their parameters from it.

ID	Title	Consumed by
SOP-SOT-001	Inventory source of truth	resolver, planner
SOP-SOT-002	Shipment & order-status source of truth	resolver
SOP-EXC-001	Inventory shortfall handling	resolver, planner
SOP-EXC-002	Duplicate orders	resolver
SOP-EXC-003	Invalid or malformed data	resolver, planner
SOP-EXC-004	Stale shipments	resolver
SOP-EXC-005	Conflicting or ambiguous records	resolver
SOP-APR-001	Action authority and approvals	resolver, planner
SOP-PLN-001	Planning eligibility and prioritization	planner
SOP-PLN-002	Capacity, skills and inventory constraints	planner
SOP-PLN-003	Replanning	planner

Full text: `POLICIES.md` .

9. Changes from the original plan

Recorded during implementation. See also `REFLECTION.md` .

1. **Planner does not read cycle counts.** It consumes the resolver's decision (order hold) and shows untriaged exceptions as warnings. This keeps triage in one place and makes the cross-agent effect observable (ORD-1004: ASSIGNED with warning in v1 → BLOCKED by EXC-2001 in v2).
2. **pickers.consumed_minutes added.** Mid-shift replanning needs remaining capacity once simulated work has been done.
3. **Per-picker sequence continues after completed work**, so unchanged assignments are reported as KEPT, not RESEQUENCED (bug found by a unit test).
4. **Two workspaces instead of six packages; raw SQL instead of an ORM.** Same boundaries, less tooling (see ARCHITECTURE.md).
5. **LLM model:** Gemini `gemini-3.1-flash-lite` (the planned 2.5 model is retired for new keys; free quota on 3.5-flash is 20 requests).
6. **Added:** a claimed-action detector (the model narrated actions before they executed) and per-run agent mode in scenario results (a green suite had hidden LLM fallbacks).
7. **Optional features (after tagging v1.0-core):** event outbox + dispatcher + deterministic shortfall detector; metrics + evaluation harness; local-search optimizer. This required splitting planner picker-choice from a finalize pass. See OPTIONAL_FEATURES.md.